

TypeScript Problem-Solving

Problem 1 — Wi-Fi Signal Status

Function name must be: **getSignalStatus**

Scenario

A router's admin dashboard shows a friendly signal status label instead of just the raw signal percentage.

Task

Create a function named `getSignalStatus`. The function receives the signal strength percentage (0–100) and returns a status string.

Status Rules

Percentage	Status
0–25	"Poor"
26–55	"Fair"
56–85	"Good"
86–100	"Excellent"

Requirements

- Accept the percentage as a number.
- Return the status as a string.

Starter Code

```
function getSignalStatus(percentage) {  
  // write your code here  
}
```

Test Cases

Input	Output
15	"Poor"
40	"Fair"
70	"Good"

95	"Excellent"
----	-------------

Problem 2 — Movie Ticket Confirmation

Function name must be: **formatTicketConfirmation**

Scenario

A cinema booking app confirms a ticket purchase with a short readable message. A ticket contains a moviegoer's name, the movie title, and the showtime.

Task

First define an appropriate type or interface for the ticket. Then create a function named `formatTicketConfirmation` that receives a ticket and returns a confirmation sentence.

Requirements

- Accept a properly typed ticket object.
- Return a string.
- Use the values from the object rather than hard-coding the result.

Starter Code

```
// TODO: define a type or interface for the ticket  
function formatTicketConfirmation(ticket){  
  // write your code here  
}
```

Test Cases

Input	Output
{ name: "Nabila", movie: "Interstellar", time: "9:15 PM" }	"Nabila's ticket for Interstellar is confirmed at 9:15 PM."
{ name: "Jamal", movie: "Inception", time: "6:00 PM" }	"Jamal's ticket for Inception is confirmed at 6:00 PM."

Problem 3 — Daily Steps Tracker

Function name must be: **calculateWeeklySteps**

Scenario

A fitness app lets a user log their step count for each day of the week as a plain number, and needs to add them all up.

Task

Create a function named `calculateWeeklySteps`. The function should receive an array of daily step counts and return the total.

Requirements

- Properly type the array of numbers.
- Return the total as a number.
- An empty array should return 0.

Starter Code

```
function calculateWeeklySteps(steps){  
  // write your code here  
}
```

Test Cases

Input	Output
[3000, 5200, 4100]	12300
[7000, 6500]	13500
[]	0

Problem 4 — Restaurant Order Total

Function name must be: **calculateOrderTotal**

Scenario

A restaurant ordering app stores each dish in the cart as an object with a name and a price, and needs to calculate the total bill.

Task

Create a function named `calculateOrderTotal`. The function should receive an array of order items and return the total price.

Order item structure: { name: string; price: number; }

Requirements

- Properly type the order item object.
- Properly type the array of items.
- Return the total as a number.
- An empty order should return 0.

Starter Code

```
// TODO: define a type for a single order item  
function calculateOrderTotal(items){  
  // write your code here  
}
```

Test Cases

Input	Output
[{name:"Burger",price:250},{name:"Fries",price:90},{name:"Soda",price:60}]	400
[{name:"Pizza",price:500}]	500
[]	0

Problem 5 — Weather Advice Picker

Function name must be: **getWeatherAdvice**

Scenario

A weather app tells the user what to do based on today's simple weather condition.

Task

First create a union type `Weather`, then create a function named `getWeatherAdvice` that receives a `Weather` and returns the correct advice.

Rules

Weather	Advice
sunny	"Wear sunscreen"
rainy	"Carry an umbrella"
cloudy	"Bring a light jacket"

TypeScript Requirement

This should produce a TypeScript error: `getWeatherAdvice("snowy")`.

Starter Code

```
function getWeatherAdvice(weather){  
    // write your code here  
}
```

Test Cases

Input	Output
"sunny"	"Wear sunscreen"
"rainy"	"Carry an umbrella"
"cloudy"	"Bring a light jacket"

Problem 6 — Employee On-Duty Finder

Function name must be: `findOnDutyEmployees`

Scenario

A staffing app stores its team as a list of employees, each with a name and whether they are currently on duty.

Task

Create a function named `findOnDutyEmployees`. The function should receive an array of employees and return only the ones currently on duty.

Employee structure: `{ name: string; onDuty: boolean; }`

Requirements

- If no employee is on duty, return an empty array.

Starter Code

```
// TODO: define a type for an employee  
function findOnDutyEmployees(employees){  
  // write your code here  
}
```

Test Cases

Input	Output
<code>[{name:"Lima",onDuty:true},{name:"Karim",onDuty:false}]</code>	<code>[{name:"Lima",onDuty:true}]</code>
<code>[{name:"Nusrat",onDuty:false}]</code>	<code>[]</code>

Problem 7 — Delivery Distance Summary

Function name must be: `getDistanceSummary`

Scenario

A delivery app wants a simple summary of a rider's trips for the day: the total distance and the average distance per trip.

Task

Create a function named `getDistanceSummary`. The function should receive an array of trip distances (in km) and return an object containing total and average.

Edge case: think about what your function should do if the distances array is empty.

Starter Code

```
function getDistanceSummary(distances){  
  // write your code here  
}
```

Test Cases

Input	Output
[4, 6, 5, 9]	{ total: 24, average: 6 }
[10, 10]	{ total: 20, average: 10 }
[]	{ total: 0, average: 0 }